

Django REST Framework (DRF) Complete Guide

Prepared for: Interns

This document provides a complete overview of Django REST Framework, including installation, setup, API examples (GET, POST, PUT, PATCH, DELETE), and JWT authentication. It is designed to serve as a practical guide for interns working on backend development with Django and REST APIs.

1. Introduction to Django REST Framework

Django REST Framework (DRF) is a powerful and flexible toolkit for building Web APIs. It simplifies creating RESTful APIs by providing serializers, authentication, permissions, and response handling. DRF integrates seamlessly with Django's ORM and views, allowing developers to expose their models as APIs efficiently.

Installation:

Run the following command to install DRF:

```
pip install djangorestframework
```

Add DRF to your INSTALLED_APPS in settings.py:

```
INSTALLED_APPS = [ ... 'rest_framework', ]
```

2. Serializers

Serializers in DRF convert Django model instances to Python data types and vice versa. They are used to render data into JSON or XML formats and validate input data when creating or updating records.

```
Example: from rest_framework import serializers from .models import Employee
class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = '__all__'
```

3. Common API Methods

Below are examples of how to implement standard CRUD operations using DRF's APIView class.

GET API

Used to retrieve data from the server.

```
class EmployeeList(APIView): def get(self, request): employees =  
Employee.objects.all() serializer = EmployeeSerializer(employees, many=True)  
return Response(serializer.data)
```

POST API

Used to create a new record in the database.

```
class EmployeeCreate(APIView): def post(self, request): serializer =  
EmployeeSerializer(data=request.data) if serializer.is_valid():  
serializer.save() return Response(serializer.data, status=201) return  
Response(serializer.errors, status=400)
```

PUT API

Used to completely update an existing record.

```
class EmployeeUpdate(APIView): def put(self, request, pk): employee =  
Employee.objects.get(id=pk) serializer = EmployeeSerializer(employee,  
data=request.data) if serializer.is_valid(): serializer.save() return  
Response(serializer.data) return Response(serializer.errors, status=400)
```

PATCH API

Used to partially update an existing record.

```
class EmployeePartialUpdate(APIView): def patch(self, request, pk): employee =  
Employee.objects.get(id=pk) serializer = EmployeeSerializer(employee,  
data=request.data, partial=True) if serializer.is_valid(): serializer.save()  
return Response(serializer.data) return Response(serializer.errors,  
status=400)
```

DELETE API

Used to delete a record from the database.

```
class EmployeeDelete(APIView): def delete(self, request, pk): employee =  
Employee.objects.get(id=pk) employee.delete() return Response(status=204)
```

4. JWT Authentication in DRF

JWT (JSON Web Token) is a compact and self-contained way to securely transmit information between parties. In DRF, JWTs are commonly used for stateless authentication where each request is verified independently.

Install JWT Authentication Library:

```
pip install djangorestframework-simplejwt
```

Add JWT settings in settings.py:

```
REST_FRAMEWORK = { 'DEFAULT_AUTHENTICATION_CLASSES': (  
    'rest_framework_simplejwt.authentication.JWTAuthentication', ), }
```

Update urls.py:

```
from rest_framework_simplejwt.views import TokenObtainPairView,  
TokenRefreshView urlpatterns = [ path('api/token/',  
TokenObtainPairView.as_view(), name='token_obtain_pair'),  
path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  
]
```

Usage Example:

1. Send a POST request to `/api/token/` with username and password to get access and refresh tokens.
2. Include the access token in headers for protected API requests: `Authorization: Bearer`
3. Use the refresh token to obtain a new access token when expired via `/api/token/refresh/`.

5. Permissions & Authentication

DRF provides built-in permission classes like `IsAuthenticated`, `IsAdminUser`, and `AllowAny`. You can also define custom permissions for fine-grained control over access to your APIs.

Example:

```
from rest_framework.permissions import IsAuthenticated
class EmployeeList(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)
```

Conclusion

Django REST Framework simplifies API development by offering a structured, secure, and maintainable architecture. With serializers, views, and JWT authentication, developers can quickly build scalable backends ready for production use.